

PROBLEM 2 : Fine-tuning nanoVLM on the Robo2VLM-1 Dataset

Rajat Vajpayee

1 Setup

Robo2VLM-1 provides five columns per row: `id` (a string encoding the source trajectory and question number), `question` (a string), `choices`, `correct_answer` (an integer index into `choices`, from 0 to 4), and `image` (84 to 640 px wide).

Model. nanoVLM-230M-8k — vision side `siglip2-base-patch16-512`, language side `SmolLM2-360M-Instruct`. The vision encoder was frozen; the projector and language model were trained (141.6M of 228.1M parameters).

Training. AdamW, lr 1.5×10^{-5} , weight decay 0.01, batch size 32, bfloat16, gradients clipped at norm 1.0, four epochs on one A100-40GB (about 75 minutes per epoch). Ten percent of the training set was held out for validation; the shipped test set (6,676 rows) was used only for final numbers.

2 Task 1: Converting the annotations into a training format

Each row gives an image, a question, a list of answer choices, and the index of the correct one. These were turned into a single text sequence that nanoVLM can be trained on, with the model writing its answer through its own language head.

- **Target.** The model predicts the option *number*, not the option text. Every answer is then a single token, so the logits at that one position give a direct probability distribution over the options. Option texts are often much longer than the expected answer. Training the model to generate the full option text would require processing many more tokens, causing longer examples to dominate the language modeling loss and skew the training objective.
- **Prompt.** An instruction, the question, the numbered options, and “Answer:”, with the image tokens placed in front. Everything except the answer token is masked during training, so the loss only ever asks for the number.
- **Parsing the choices.** The `choices` field is a Python list saved as a string with single quotes, so `json.loads` fails on it. `ast.literal_eval` reads it correctly.
- **Four or five options.** Rows have either four or five choices. The prompt includes only the available options. Any missing options are masked when computing probabilities. So the model is evaluated only on the options it was given.
- **Image tiles.** nanoVLM upsamples images to 2048 first and then splits them into multiple tiles, each of size (512x512). Since the Robo2VLM-1 images are under 512x512, this created many unnecessary tiles containing only enlarged versions of the same image. Reducing the maximum image size to 512 ensures that each image is processed as a single tile, greatly reducing computation and training time without losing any visual information.
- **Answer position.** “1” tokenises as two tokens, a space followed by the digit. The position used for scoring had to be shifted past the space, otherwise the reported confidence would be about whitespace rather than about the answer.

Look at the image and answer the multiple-choice question. Respond with the number of the correct option only.

Question: Is there any obstacle blocking the robot from reaching bottle?

Options: 1. No 2. Cannot be determined 3. Partially reachable 4. Yes

Answer:

→ target: 1

3 Task 2a and 2d: Evaluation metrics and Baseline Comparison

How the model is scored. At the position that predicts the answer, I take the logits, keep only the ones for the option numbers, mask out any options that row does not have and normalise. Perplexity is computed separately over the model’s full vocabulary.

Why these metrics.

- *Accuracy.* The obvious one, but it starts from a 20% floor, and always picking the commonest answer already gives 22.9%.
- *Macro precision, recall and F1.* Averaged over the five options. These show whether the model does equally well on all of them.

- *Cohen’s kappa*. Agreement after subtracting the agreement you would get by chance. This separates learning the task from learning which answer is most common.
- *Perplexity*. The exponential of the average surprise on the correct token. It is the only metric that reflects how confidently the model got there rather than just whether it was right. A model that knows the format but nothing about the task would sit near 5.
- *ECE*. When the model says it is 90% sure, is it right 90% of the time? This matters for a robot that needs to know when it is unsure. Mean confidence is reported next to it so we can tell whether the model is too confident or not confident enough.
- *Mass outside the option numbers*. How much probability lands on tokens that are not answers. Near zero means the output format is learned, which is what lets us read perplexity as uncertainty about the answer rather than confusion about what to write.

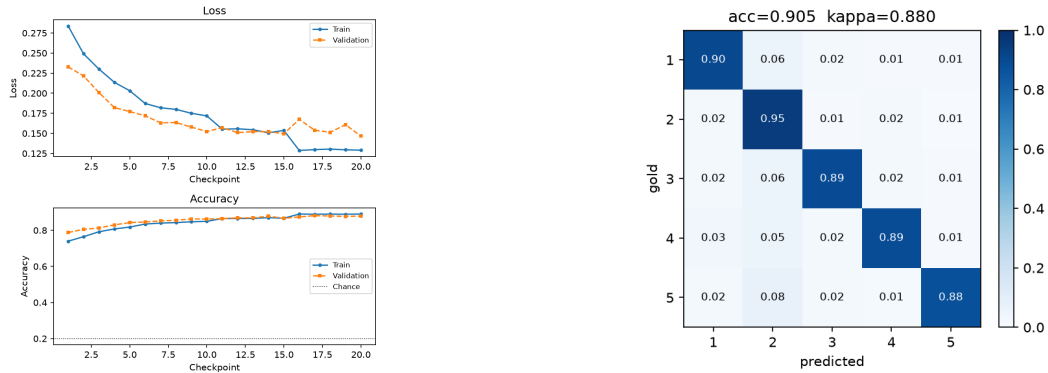
Table 1: Test set results on all 6,676 examples. The baseline uses the original pre-trained Hugging Face nanoVLM checkpoint with the same architecture, while the fine-tuned model is trained on the Robo2VLM-1 dataset.

Metric	Baseline	After 1 epoch	Final (4 epochs)
Accuracy	0.2380	0.8292	0.9046
Macro precision	0.2419	0.8395	0.9107
Macro recall	0.2342	0.8262	0.9028
Macro F1	0.2312	0.8307	0.9050
Cohen’s kappa	0.0390	0.7844	0.8797
Perplexity	6.1464	1.4684	1.2533
ECE	0.2315	0.0121	0.0096
Mean confidence	0.4696	0.8234	0.9014
Mass outside options	0.0209	0.000088	0.000044

Accuracy went from 23.8% to 90.5%. Kappa went from 0.039, which is no better than guessing, to 0.880. The kappa number is the more useful one here because it shows the model did not just learn which answers tend to be correct.

Perplexity fell from 6.15 to 1.25. Since the mass outside the option numbers is 0.000044, the model is putting almost all of its probability on the answer options, so 1.25 reflects confidence in the right answer rather than uncertainty about what to write.

ECE dropped from 0.23 to 0.0096, with mean confidence at 0.901 against accuracy 0.905. Fine-tuning often makes a model more accurate and more overconfident at the same time. That did not happen here.



(a) Training and validation loss and accuracy.

(b) Confusion matrix with row normalisation.

Figure 1: Training results after fine-tuning. Validation accuracy improves during the first three epochs before plateauing. The widening gap between training and validation loss suggests the onset of overfitting, so training was stopped after the fourth epoch.

3.1 Does the model actually need the image?

Several templates were scoring exactly 100% on hundreds of examples, which looked too good. So I ran the whole test set again with the image replaced by zeros, keeping the text identical.

Table 2: Accuracy with the real image against a blank image, by template.

Template (first words of question)	n	With image	Blank	Drop
the robot’s task is to...	856	1.000	1.000	0.000
is there any obstacle blocking...	543	1.000	1.000	0.000
the robot task is to put...	216	1.000	1.000	0.000
which language instruction best describes...	147	1.000	1.000	0.000
(four more templates, 534 rows)	534	1.000	1.000	0.000
the robot is tasked to pick...	165	0.921	0.412	−0.509
in the image from ext... which	934	0.854	0.365	−0.489
is the robot’s gripper open...	550	0.929	0.682	−0.247
is the robot’s grasp of...	299	0.833	0.609	−0.224
the robot is to put the...	114	0.912	0.868	−0.044
in the left image (ext camera)...	349	0.321	0.344	+0.023
Overall	6676	0.9046	0.7509	−0.154

With no image at all, accuracy falls from 90.5% to 75.1% and kappa from 0.880 to 0.686. Similar behaviour has also been observed in MIRAGE,¹ which reports that vision-language models can often answer many questions using language priors even when visual information is absent. The templates fall into three groups.

About 2,300 rows, roughly a third of the test set, score exactly 100% with a blank image. These are mostly instruction-matching questions where the question text and the correct option use the same words, so the model can match the strings without looking at anything. The gripper and grasp questions lose 22 to 51 points when blinded, so those really are using the image. And ext camera template sits at the bottom, either way the model never learned to use the image for it at all.

This does not make the 90.5% wrong, but it changes what it measures. Part of the score comes from a text shortcut in how the dataset is built and the headline number should be read with that in mind.

4 Task 2b: Attention and saliency maps

nanoVLM is decoder-only. The image becomes 64 tokens at the front of the sequence and image and text then go through the same transformer. So how much the answer looked at patch j is just an entry in the attention matrix. Getting that matrix took some work, since nanoVLM does not expose attention weights and the fused kernels never build the probability matrix. I wrap PyTorch’s attention function for one forward pass and recompute the softmax.

Grad-CAM answers a different question. Not what the model looked at but what changed the answer. I take the gradient of the correct answer’s score with respect to the image token embeddings, multiply by the embeddings and keep the positive part.

The two maps disagree and Grad-CAM is the one that follows the question. In the example below the question asks which coloured point is farthest from the camera. Grad-CAM covers the back of the scene where the blue and purple points sit. Attention rollout picks the top-left corner instead.

The attention matrix shows why. Nearly all the weight goes to one or two image tokens, the same ones whatever the question is. This is normal in decoder-only transformers: attention has to sum to one and the first token is the one every later position can see, so the model parks unneeded attention there. What is left over the other patches is too small to separate from noise. The prompt words do not attend to any specific patch either, so the visual information must be reaching the answer through the residual stream. Attention maps cannot show that path but gradients follow it, which is why Grad-CAM still works. (see Figure 2)

5 Task 2c: Sampling strategies

All four are written from scratch as filters on the logits, applied in this order: temperature, top- k , top- p , min- p . Temperature divides the logits. Below 1 it sharpens the distribution and above 1 it flattens it, but it never removes a token. Top- k keeps the k best tokens, so the width is fixed whether the model is sure or not. Top- p keeps the smallest set of tokens whose probabilities add up to p . Min- p keeps tokens that are at least p times as likely as the top one, so the threshold moves with the peak.

¹<https://arxiv.org/abs/2603.21687>

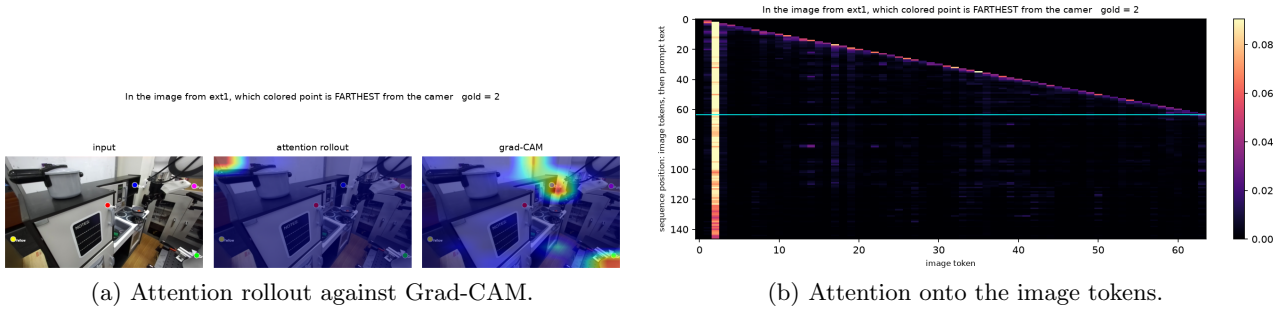


Figure 2: The question asks which coloured point is farthest from the camera. Grad-CAM covers the back of the scene. Attention rollout sits in the top-left corner, which has nothing to do with the question. The matrix shows the reason: almost all the weight is in one or two columns. Both figures come from a fixed sample of test examples so the baseline and fine-tuned runs are comparable.

Table 3: Effect of each strategy. 480 test examples, 3 seeds. *Valid* means the output was a legal option number. *Consist.* means the three seeds agreed. *Kept* is how many tokens survived the filter. Temperature removes none, so it shows the full vocabulary.

Strategy	Acc.	Valid	Consist.	Kept	Strategy	Acc.	Valid	Consist.	Kept
Greedy	0.900	1.000	1.000	1.0	top- $p=0.5$	0.901	1.000	0.988	1.1
top- $k=1$	0.900	1.000	1.000	1.0	top- $p=0.9$	0.890	1.000	0.952	1.7
$T=0.5$	0.897	1.000	0.967	49218	top- $p=0.99$	0.882	1.000	0.942	3.4
$T=1.0$	0.881	1.000	0.942	49218	min- $p=0.05$	0.885	1.000	0.947	1.8
$T=1.5$	0.865	0.997	0.922	49218	min- $p=0.1$	0.887	1.000	0.952	1.6
top- $k=5$	0.884	1.000	0.944	5.0	min- $p=0.1, T=2$	0.849	1.000	0.924	2.5
top- $k=50$	0.881	1.000	0.942	50.0					

Top- $k=1$ produces the same results as greedy decoding and greedy gives identical outputs across all three random seeds, confirming that the implementation behaves as expected.

Greedy decoding achieves the highest accuracy and consistency, while increasing the sampling temperature or using wider sampling strategies reduces both. For example, increasing the temperature from 0.5 to 1.5 lowers accuracy by about 3 percentage points and consistency by about 4.5 percentage points.

6 What I would do differently

- **Split by trajectory, not by row.** The *id* field identifies the source trajectory for each question. Splitting the data by trajectory rather than by individual rows would provide a more reliable validation set by preventing information leakage between training and validation. Given more time, I would also construct a balanced validation set with an equal number of examples from each question category.
- **Unfreeze the vision encoder.** I never did. If the ceiling is what the model can see rather than how it reasons, that is where the gain is.
- **Use a learning rate schedule.** The gradient norm rose above the clipping threshold of 1.0, peaking near 2.3, so the step size was being scaled down automatically. Better to do that on purpose using lr-schedule.